HW10

zhijing huang

2026-11-01

Question 1

a. The random seed is 43210. The two figures below compare average squared bias and variance across the tuning settings.

```
library(RLT)
```

```
## Reinforcement Learning Trees (RLT) v3.2.6
## Experimental version at https://github.com/teazrq/RLT
```

```
bootstrap_arg <- if ("resample.replace" %in% names(formals(RLT::RLT))) {
  "resample.replace"
} else {
  "replacement"
}
set.seed(43210)
Sigma <- matrix(0.3, nrow = 5, ncol = 5)
diag(Sigma) <- 1

draw_x <- function(n) {
  matrix(rnorm(n * 5), nrow = n) %*% chol(Sigma)
}
true_f <- function(x) {
  rowSums(x[, c(1, 3, 5), drop = FALSE])
}

x_test <- draw_x(200)
f_test <- true_f(x_test)

settings <- unique(rbind(
  data.frame(nmin = c(1, 5, 10, 20, 30), mtry = 3),
  data.frame(nmin = 10, mtry = 1:5)
))
rownames(settings) <- NULL

predictions <- array(
  NA_real_,
  dim = c(200, 100, nrow(settings))
)
for (b in 1:100) {
```

```r
  x_train <- draw_x(200)
  y_train <- true_f(x_train) + rnorm(200)

  for (s in seq_len(nrow(settings))) {
    fit_args <- list(
      x = x_train,
      y = y_train,
      model = "regression",
      ntrees = 500,
      nmin = settings$nmin[s],
      mtry = settings$mtry[s],
      resample.prob = 1,
      reinforcement = FALSE
    )
    fit_args[[bootstrap_arg]] <- TRUE
    if (bootstrap_arg == "replacement") {
      # Older RLT prints an OpenMP notice for every fit; it is not an error.
      invisible(capture.output(fit <- do.call(RLT::RLT, fit_args)))
    } else {
      fit <- do.call(RLT::RLT, fit_args)
    }

    predictions[, b, s] <- predict(fit, x_test)$Prediction
  }
  if (b %% 10 == 0) message("Completed ", b, " of 100 repetitions")
}
```

```
## Completed 10 of 100 repetitions

## Completed 20 of 100 repetitions

## Completed 30 of 100 repetitions

## Completed 40 of 100 repetitions

## Completed 50 of 100 repetitions

## Completed 60 of 100 repetitions

## Completed 70 of 100 repetitions

## Completed 80 of 100 repetitions

## Completed 90 of 100 repetitions

## Completed 100 of 100 repetitions
```

```r
# Average predictions and their across-repetition variances at each test point.
```

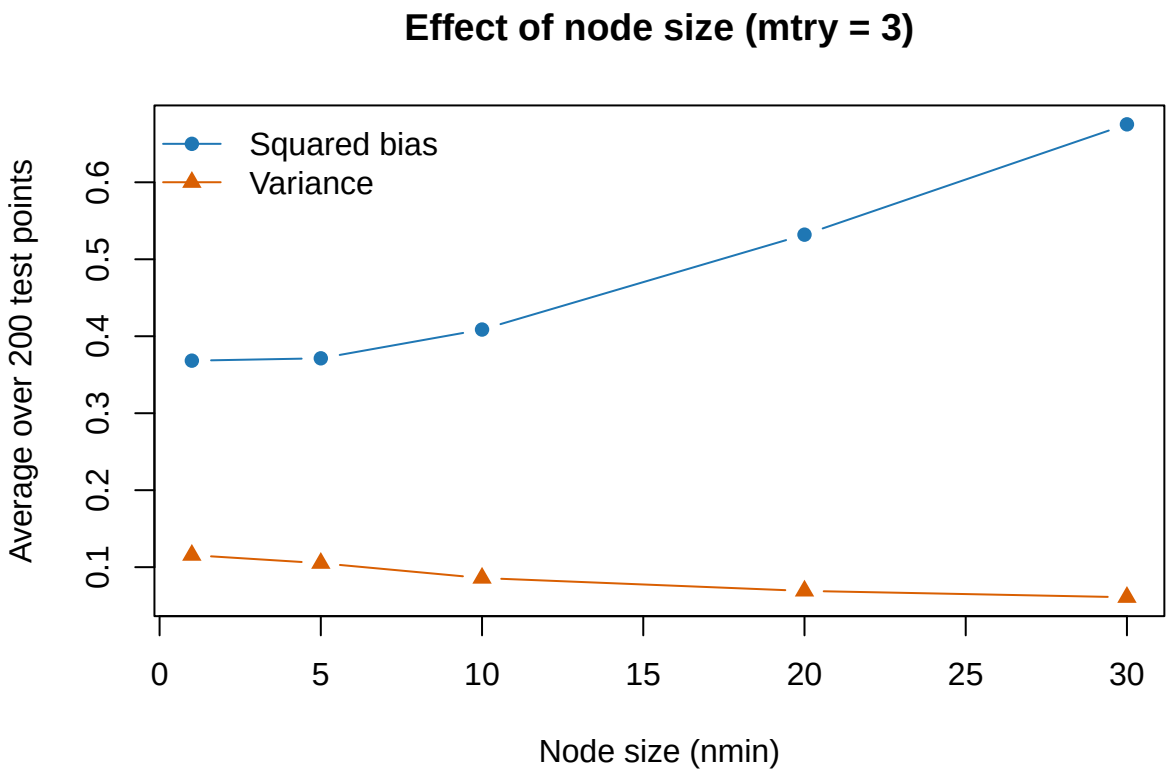
```r
mean_prediction <- apply(predictions, c(1, 3), mean)
point_variance <- apply(predictions, c(1, 3), var)
results <- data.frame(
  settings,
  squared_bias = colMeans(sweep(mean_prediction, 1, f_test, "-")^2),
  variance = colMeans(point_variance)
)
results
```

```
##   nmin mtry squared_bias   variance
## 1    1    3    0.3682707 0.11562300
## 2    5    3    0.3713214 0.10517057
## 3   10    3    0.4086905 0.08587297
## 4   20    3    0.5319468 0.06915382
## 5   30    3    0.6752647 0.06093084
## 6   10    1    1.0971978 0.04317944
## 7   10    2    0.5482949 0.06648895
## 8   10    4    0.3504076 0.10380298
## 9   10    5    0.3203339 0.11880619
```

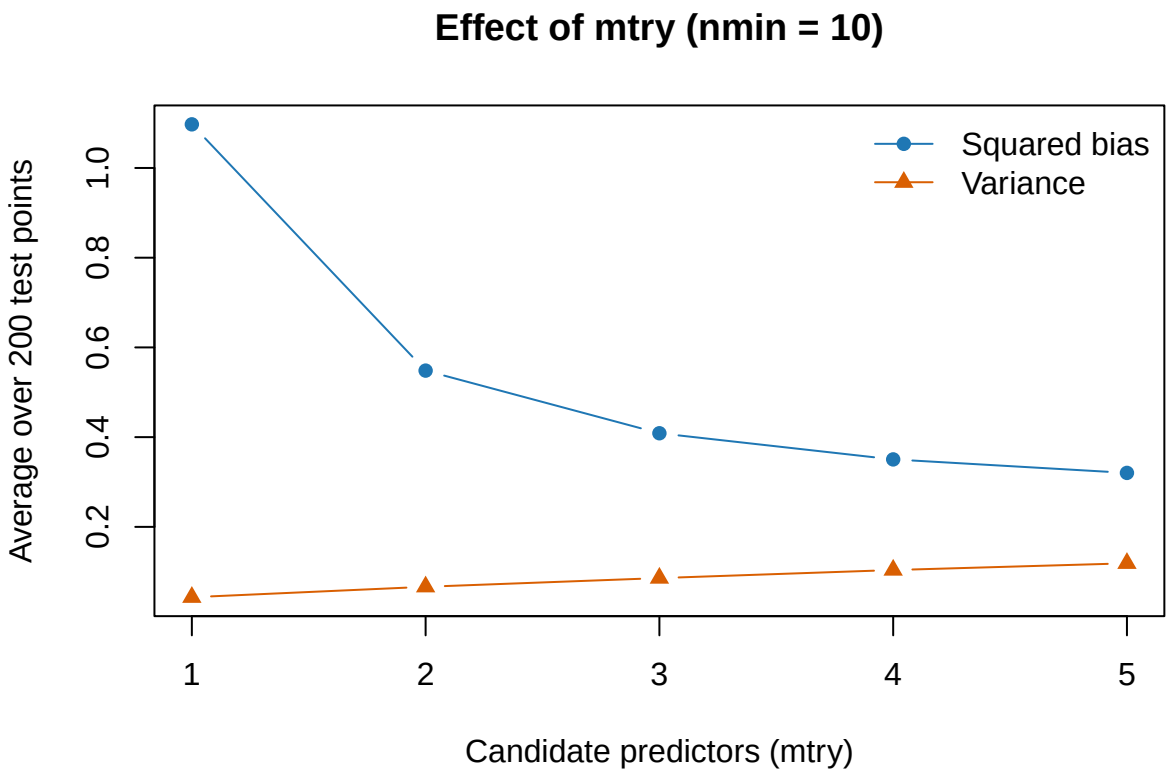
```r
node_1 <- subset(results, nmin == 1 & mtry == 3)
node_30 <- subset(results, nmin == 30 & mtry == 3)
mtry_1 <- subset(results, nmin == 10 & mtry == 1)
mtry_5 <- subset(results, nmin == 10 & mtry == 5)

plot_tradeoff <- function(rows, x_name, x_label, main_title) {
  rows <- rows[order(results[[x_name]][rows])]
  values <- as.matrix(results[rows, c("squared_bias", "variance")])
  matplot(results[[x_name]][rows], values, type = "b", pch = c(16, 17),
          lty = 1, col = c("#1f77b4", "#d95f02"),
          xlab = x_label, ylab = "Average over 200 test points",
          main = main_title)
  legend(if (x_name == "nmin") "topleft" else "topright",
         legend = c("Squared bias", "Variance"),
         col = c("#1f77b4", "#d95f02"), pch = c(16, 17), lty = 1,
         bty = "n")
}

plot_tradeoff(which(results$mtry == 3), "nmin", "Node size (nmin)",
              "Effect of node size (mtry = 3)")
```




```
plot_tradeoff(which(results$nmin == 10), "mtry", "Candidate predictors (mtry)",
              "Effect of mtry (nmin = 10)")
```



b. A large node size stops the trees earlier, so the fit is less flexible. Here, increasing `nmin` from 1 to 30 (with `mtry = 3`) raises average squared bias from about 0.368 to 0.675, while lowering average variance from about 0.116 to 0.061. The smoother predictions vary less between training samples, but they miss more of the true function's pattern.

At `nmin = 10`, increasing `mtry` from 1 to 5 makes the three relevant predictors more likely to be considered at each split. In this simulation, average squared bias falls from about 1.097 to 0.32. The drawback is that trees become more similar and the forest's predictions vary more across training samples: average variance rises from about 0.043 to 0.119. Thus a larger `mtry` can reduce bias but increase variance; the balance depends on the data.

If each tree drew only 100 observations with replacement, I would expect the preferred node size to shift smaller. With fewer sampled observations, a tree reaches a fixed node-size cutoff sooner, so reducing `nmin` may preserve useful splits. Smaller nodes could also raise variance, so this is an expectation to check with the 100-observation simulation rather than a guaranteed result.

Question 2

a.

```
data_file <- if (file.exists("data/mnist-first2000.csv.gz")) {
  "data/mnist-first2000.csv.gz"
} else {
  "week-10/data/mnist-first2000.csv.gz"
}
```

```r
mnist <- read.csv(data_file)
pixel_cols <- setdiff(names(mnist), "Digit")

train_digits <- mnist[1:1000, ]
train_digits <- train_digits[train_digits$Digit %in% c(1, 6, 7), ]
test_digits <- mnist[1001:2000, ]
test_digits <- test_digits[test_digits$Digit %in% c(1, 6, 7), ]

pca <- prcomp(train_digits[, pixel_cols], center = TRUE, scale. = FALSE)
x_train_pc <- pca$x[, 1:20, drop = FALSE]
x_test_pc <- predict(pca, newdata = test_digits[, pixel_cols])[, 1:20, drop = FALSE]
y_train_digit <- factor(train_digits$Digit, levels = c(1, 6, 7))
y_test_digit <- factor(test_digits$Digit, levels = c(1, 6, 7))

set.seed(43210)
bootstrap_arg <- if ("resample.replace" %in% names(formals(RLT::RLT))) {
  "resample.replace"
} else {
  "replacement"
}
fit_args <- list(x = x_train_pc, y = y_train_digit,
                 model = "classification", ntrees = 1000,
                 resample.prob = 1, reinforcement = FALSE)
fit_args[[bootstrap_arg]] <- TRUE
if (bootstrap_arg == "replacement") {
  fit_args$track.obs <- TRUE
  invisible(capture.output(forest_default <- do.call(RLT::RLT, fit_args)))
  tree_predictions <- predict(forest_default, x_train_pc)$AllPrediction
  oob_votes <- matrix(0, nrow(x_train_pc), nlevels(y_train_digit))
  for (tree in seq_len(forest_default$ntrees)) {
    oob_id <- which(forest_default$ObsTrack[, tree] == 0)
    oob_votes[oob_id, ] <- oob_votes[oob_id, ] +
      tree_predictions[[tree]][oob_id, , drop = FALSE]
  }
  stopifnot(all(rowSums(forest_default$ObsTrack == 0) > 0))
  oob_class <- factor(levels(y_train_digit)[max.col(oob_votes)],
                      levels = levels(y_train_digit))
  oob_error <- mean(oob_class != y_train_digit)
} else {
  forest_default <- do.call(RLT::RLT, fit_args)
  oob_error <- as.numeric(forest_default$Error)
}

cat("Training images:", nrow(x_train_pc), "\n")
```

```
## Training images: 327
```

```r
cat("Test images:", nrow(x_test_pc), "\n")
```

```
## Test images: 317
```

```r
cat("OOB misclassification rate:", round(oob_error, 4), "\n")
```

```
## OOB misclassification rate: 0.0306
```

The default forest's out-of-bag misclassification rate is 0.0306. Each training image is evaluated using only trees whose bootstrap samples omitted that image; the held-out test images are not used for this estimate.

b. The OOB errors for the candidate settings range from about 0.021 to 0.043. The selected setting is `mtry = 3` and `nmin = 10`, with the lowest OOB error. PC2 has the highest permutation importance (about 0.684), while PC15 has the lowest (about 0.024). Permuting PC2 disrupts OOB predictions the most; PC15 contributes less to this fitted forest.

```r
grid <- expand.grid(
  mtry = c(3, 6, 10, 15),
  nmin = c(2, 5, 10, 20)
)
grid$OOB_error <- NA_real_
oob_rate_old <- function(fit, x, y) {
  tree_prob <- predict(fit, x)$AllPrediction
  votes <- matrix(0, nrow(x), nlevels(y))

  for (t in seq_len(fit$ntrees)) {
    oob_id <- which(fit$ObsTrack[, t] == 0)
    votes[oob_id, ] <- votes[oob_id, ] +
      tree_prob[[t]][oob_id, , drop = FALSE]
  }

  stopifnot(all(rowSums(fit$ObsTrack == 0) > 0))
  predicted <- factor(levels(y)[max.col(votes)], levels = levels(y))
  mean(predicted != y)
}
best_error <- Inf
best_fit <- NULL

for (i in seq_len(nrow(grid))) {
  set.seed(43210 + i)

  args <- list(
    x = x_train_pc,
    y = y_train_digit,
    model = "classification",
    ntrees = 1000,
    mtry = grid$mtry[i],
    nmin = grid$nmin[i],
    resample.prob = 1,
    reinforcement = FALSE,
    importance = TRUE
  )
  args[[bootstrap_arg]] <- TRUE

  if (bootstrap_arg == "replacement") {
    args$track.obs <- TRUE
```

```
    invisible(capture.output(fit <- do.call(RLT::RLT, args)))
    grid$OOB_error[i] <- oob_rate_old(fit, x_train_pc, y_train_digit)
  } else {
    args$seed <- 43210 + i
    fit <- do.call(RLT::RLT, args)
    grid$OOB_error[i] <- as.numeric(fit$Error)
  }

  if (grid$OOB_error[i] < best_error) {
    best_error <- grid$OOB_error[i]
    best_fit <- fit
  }
}
grid[order(grid$OOB_error), ]
```

```
##    mtry nmin  OOB_error
## 9     3   10 0.02140673
## 1     3    2 0.02446483
## 2     6    2 0.03058104
## 5     3    5 0.03058104
## 6     6    5 0.03058104
## 4    15    2 0.03363914
## 7    10    5 0.03363914
## 8    15    5 0.03363914
## 10    6   10 0.03363914
## 11   10   10 0.03363914
## 14    6   20 0.03363914
## 13    3   20 0.03669725
## 15   10   20 0.03669725
## 3    10    2 0.03975535
## 12   15   10 0.03975535
## 16   15   20 0.04281346
```

```
grid[which.min(grid$OOB_error), ]
```

```
##   mtry nmin  OOB_error
## 9    3   10 0.02140673
```

```
importance_table <- data.frame(
  PC = colnames(x_train_pc),
  importance = as.numeric(best_fit$VarImp)
)
importance_table[order(-importance_table$importance), ]
```

```
##      PC importance
## 2   PC2 0.68401901
## 1   PC1 0.48695941
## 3   PC3 0.11253410
## 5   PC5 0.10594703
## 6   PC6 0.09887612
## 7   PC7 0.07653006
```

```
## 11 PC11 0.06487850
## 4   PC4 0.06097733
## 18 PC18 0.05414988
## 12 PC12 0.05253755
## 10 PC10 0.04673000
## 13 PC13 0.04273825
## 14 PC14 0.04015835
## 8   PC8 0.03883576
## 9   PC9 0.03617466
## 16 PC16 0.03418816
## 20 PC20 0.03362500
## 17 PC17 0.02983186
## 19 PC19 0.02476231
## 15 PC15 0.02403383
```

c.

```
test_output <- predict(best_fit, x_test_pc)
raw_prediction <- test_output$Prediction

if (is.numeric(raw_prediction) &&
    all(raw_prediction %in% 0:(nlevels(y_test_digit) - 1))) {
  test_predicted <- factor(
    best_fit$ylevels[raw_prediction + 1],
    levels = levels(y_test_digit)
  )
} else {
  test_predicted <- factor(
    as.character(raw_prediction),
    levels = levels(y_test_digit)
  )
}

test_error <- mean(test_predicted != y_test_digit)
confusion <- table(Actual = y_test_digit, Predicted = test_predicted)

test_error
```

```
## [1] 0.0126183
```

```
confusion
```

```
##       Predicted
## Actual   1   6   7
##      1 102   0   2
##      6   1 105   0
##      7   0   1 106
```

Question 3

a. Because the aim is for prediction, it is important for the forest to produce a general range of prediction based on the original dataset, which later could be applied in new datasets. The most important rule is not to underfit or overfit, and dimensionality couldn't be too complicated to apply the bootstrap method.

b. My proposed instructions, ordered from basic analysis steps to presentation preferences, are:

1. Identify the response, predictors, prediction task, and evaluation metric before fitting.
2. Check the sample size, number of predictors, missing values, and variable types.
3. Split the data into training and testing portions before tuning.
4. Fit a baseline random forest and record its settings.
5. Compare a manageable set of tuning settings with the baseline forest.
6. Select the model using OOB error or cross-validation, without using test outcomes.
7. Use bootstrap sampling to build trees; address high dimensionality through predictor handling and `mtry`.
8. Keep observations when possible, and report any exclusions or missing-data handling.
9. Organize the model comparisons into a clear table.
10. Use simple language in the report, and state limitations or possible sources of error.